

Refactoring 개요

CONTENTS

목차

Chapter1

Refactoring 개념

Chapter2

리팩토링 Quiz

Chapter3

리팩토링 Example

Chapter4

자주 사용되는 Refactoring 기법

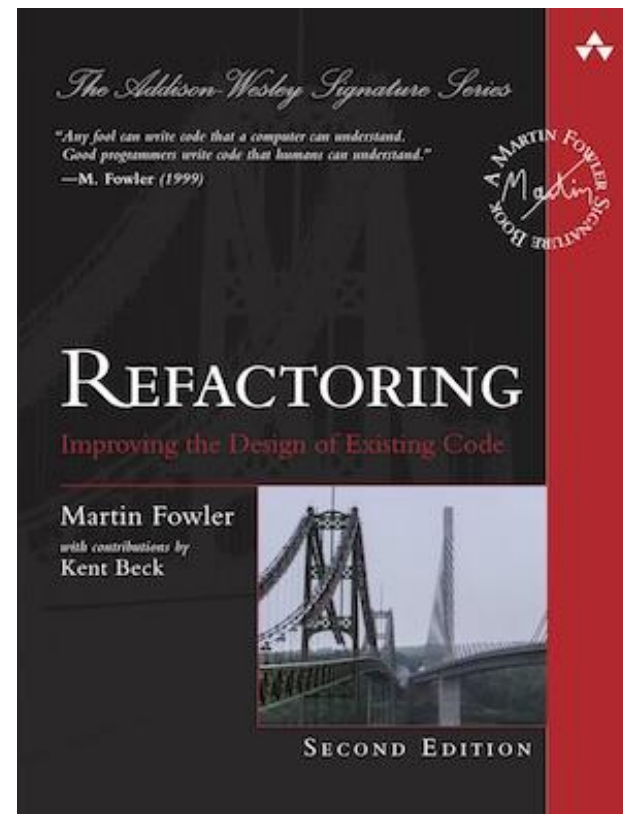
Chapter5

리팩토링시 고려할 점

Chapter1

Refactoring 개념

효율적인 코드 최적화를 위한 리팩토링



리팩토링이란?

새로운 소스코드, 새로운 기능을 만들어 내는 것이 아니다.
외부 동작은 변경하지 않고, 내부 구조를 변경하는 작업이다.

입력에 대한 출력값이 동일하게 유지되면서
내부 소스코드를 Clean하게 개선하는 작업이 리팩토링이다.

리팩토링의 시작

1. 켄트 벡

XP 개념을 저서에서 발표 (1999)

Refactoring 개념 등장

2. 마틴 파울러 + 켄트벡

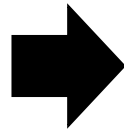
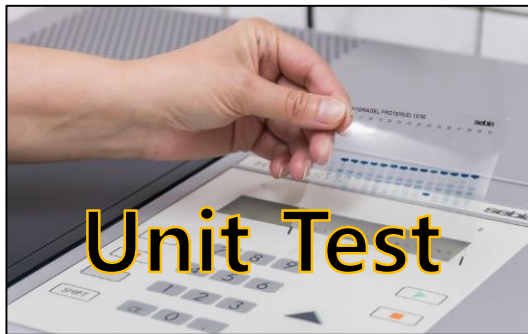
마틴 파울러는 켄트벡의 리팩토링을 보고, 반했음

Refactoring 의 개념을 정리하여, 책을 냈음 (2000)

리팩토링은 Unit Test가 준비되어야 한다.

테스트를 통해, 입출력 결과가 바뀌지 않음을 검증할 수 있어야 한다.

- ✓ 리팩토링 작업 후 기능이 바뀌었는지 확인이 가능
- ✓ 리팩토링 도중 발생하는 실수를 잡을 수 있음
- ✓ IDE 리팩토링 도구를 믿지않고, 지속적인 테스트 필요



유닛테스트로
입출력 불변을 검증

왜 리팩토링을 해야 하는가?

기존에 잘 동작하는 코드임에도 불구하고 리팩토링을 해야하는 이유

1. 프로젝트 진행속도

나쁜 코드 위에는 나쁜 코드가 얹어지게 되고,
개선이 없다면 점차 설계가 복잡해지면서 프로젝트의 속도는 점차 느려지게 된다.

2. 추가되는 요구사항에 맞지 않는 설계

과거에는 좋았던 설계라도,
요구사항이 추가되면서 기존 설계와 맞지 않게 된다.

Chapter2

리팩토링 Quiz

마틴파울러의 리팩토링에 대한 Idea Quiz.



버그 수정은 리팩토링 작업에 포함된다.
(O, X)

A1. 버그 수정은 리팩토링에 포함 안함

답 : X

버그를 수정하면 출력 결과가 바뀌게 된다.
따라서, 버그 수정은 리팩토링 작업에 포함되지 않는다.

버그 수정과 리팩토링은 별개 작업이며,
동시에 하지 않는 것을 권장한다.

기능 추가 구현과 동시에 리팩토링을 하는 것은
시간을 절약할 수 있는 좋은 방법이라고 소개한다.
(O, X)

- 리팩토링 책 저자, 마틴파울러 기준

A2. 리팩토링할 때 기능 추가 권장 안함

답은 X

마틴 파울러는 리팩토링과 기능 추가를 동시에 하는 것은 권장하지 않는다.

1. 먼저, 기능 추가와 리팩토링은 별개 작업이다.
기능 추가는 출력결과가 바뀌기 때문
2. 리팩토링과 기능 추가를 동시에 하는 경우, 문제 발생시
리팩토링으로 인한 문제인지 기능추가로 인한 문제인지 원인 찾기가 어려워진다.
3. 기능을 추가하기 전, 리팩토링을 먼저 하는 것을 추천한다.
리팩토링을 하면 가독성이 좋고 유지보수 하기 좋아지기 때문에
보다 기능추가하기 수월해진다.

**프로젝트 초기에 아키텍트의 설계를 완벽히 하면,
리팩토링이 필요 없다.**

(O, X)

A3. 기능추가가 있는 한, 리팩토링은 필요하다.

답은 **X**

초기 상황에서 완벽한 설계라고 해도,
기능 추가와 버그 수정들을 하다보면 **설계는 점차 무너진다.**

따라서 리팩토링은 지속적으로 필요하다.

리팩토링은 개발자마다 Clean한 코드의 기준이 다르다.
따라서, 리팩토링의 결과는 개발자마다 다를 수 있다.

(O, X)

Q4. 리팩토링의 결과는 주관적이다.

답은 O

개발자마다 주어진 환경, 고객사 요구조건 예상치, 개발자의 관점에 따라 주관적이다.

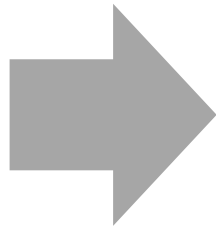
클린 코드 판별 기준 : 객관적 < **주관적**

“무조건 이렇게 코드 작성해야 한다” 는 없다.

따라서 리팩토링 작업 또한 자신의 논리적 판단을 근거로 코드를 수정하는 작업

깔끔한 방 정리

심한 경우는
누가 봐도, 정리가 된 코드와 정리가 안된 코드가 구분이 된다.
소스코드도 마찬가지이다.

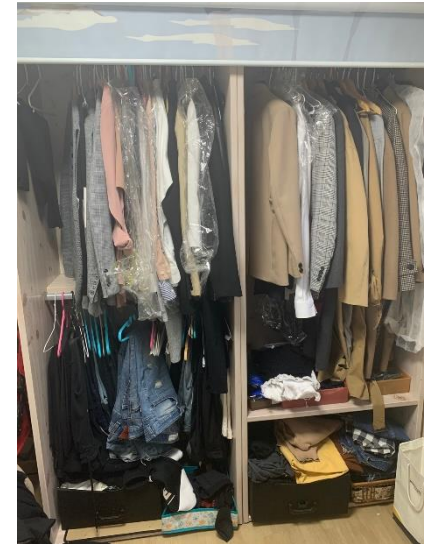
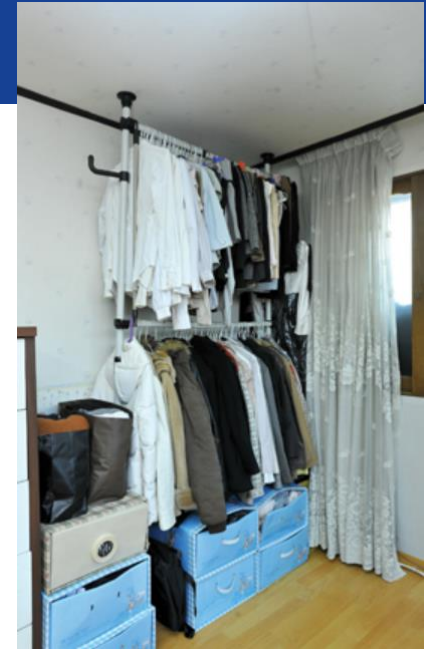
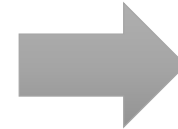
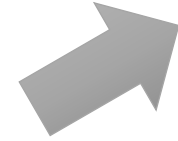


선택적인 방 정리 방법

어떻게 정리할 것인지에 대해서는, 방법의 차이가 크다.

미래 예측 : 차후 옷을 더 많이 구매가 될지 / 버릴지

사용성 판단 : 매일 꺼내는 옷인지, 가끔 꺼내는 옷인지



리팩토링은 주관적이다.

누구나 공감하는 리팩토링 필요성

중복적이고, 유닛 테스트가 어렵고, 누가 봐도 가독성이 떨어지는 코드

주관적 판단

상대방이 한 리팩토링이 마음에 들수도, 아닐수도 있음

잘못된 생각

Switch는 무조건 지저분한 코드이며, 다형성으로 변경한 코드가 깔끔한 코드다. (X)

올바른 가이드

Switch문을 다형성을 이용한 구현으로 변경했을 때, 가독성이 향상될지 각자의 고민하고 결정 (O)

```
System.out.print("num= ");
char input = scanner.next().charAt(0);
int num = input - '0';

switch (num) {
    case 0:
        System.out.println("서울");
        break;
    case 1:
        System.out.println("부산");
        break;
    case 2:
        System.out.println("광주");
        break;
    case 3:
        System.out.println("대전");
        break;
    default:
        System.out.println("지사 등록을 해야합니다.");
}
```

리팩토링의 정의

리팩토링은 코드를 개선하는 작업이다. 단, 출력 결과에 변화가 없어야 한다.

리팩토링을 하기 전, 테스트코드가 필요하다.

출력 결과에 변경이 없는지 확인할 수 있는 테스트를 준비해야 한다.

리팩토링을 통한 코드 품질 개선활동을 안전하게 할 수 있도록 한다.

리팩토링은 주관적이다

어떻게 리팩토링을 할지, 정답이 있는 것이 아니다. 개발자들마다 생각이 다를 수 있다.

가독성, 유지보수성, 팀내 코딩스타일 등을 고려해서 결정해야 한다.

Chapter3

리팩토링 Example

SplitAndSum 소스코드 분석하기

해당 소스코드를 다운로드 후 빌드,
소스코드를 이해해보자.

```
public class StringCalculator {  
  
    public static int splitAndSum(String text) {  
        int result = 0;  
  
        if (text == null || text.isEmpty()) {  
            result = 0;  
        }  
        else {  
            String[] values = text.split("-");  
            for (String value : values) {  
                result += Integer.parseInt(value);  
            }  
        }  
  
        return result;  
    }  
}
```

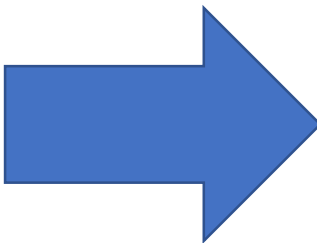
Text가 비었으면 리턴하고,
Text를 Split 후 Int로 바꾸고 Sum을 구한다.

Code Readability

리팩토링으로 코드 가독성을 더 높이자.

- 코드 가독성 : 소스코드를 논리적으로 이해하기 위한 정도를 나타냄

```
public class StringCalculator {  
  
    public static int splitAndSum(String text) {  
        int result = 0;  
  
        if (text == null || text.isEmpty()) {  
            result = 0;  
        }  
        else {  
            String[] values = text.split("-");  
            for (String value : values) {  
                result += Integer.parseInt(value);  
            }  
        }  
  
        return result;  
    }  
}
```



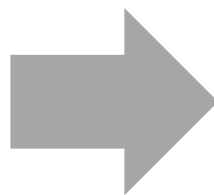
```
public static int splitAndSum(String text) {  
  
    if (isEmpty(text)) return 0;  
    return getSum(toInts(split(text)));  
}
```

Text가 비었으면 리턴하고,
Text를 Split 후 Int로 바꾸고 Sum을 구한다.

한 단계의 들여쓰기를 한다.

함수로 빼서, 들여쓰기를 없앤다.

```
public class StringCalculator {  
  
    public static int splitAndSum(String text) {  
        int result = 0;  
  
        if (text == null || text.isEmpty()) {  
            result = 0;  
        }  
        else {  
            String[] values = text.split("-");  
            for (String value : values) {  
                result += Integer.parseInt(value);  
            }  
        }  
  
        return result;  
    }  
}
```



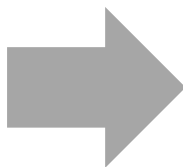
```
public class StringCalculator {  
  
    public static int splitAndSum(String text) {  
        int result = 0;  
  
        if (text == null || text.isEmpty()) {  
            result = 0;  
        }  
        else {  
            String[] values = text.split("-");  
            result += getSum(values);  
        }  
  
        return result;  
    }  
  
    private static int getSum(String[] values) {  
        int sum = 0;  
        for (String value : values) {  
            sum += Integer.parseInt(value);  
        }  
        return sum;  
    }  
}
```


else를 없앤다.

return 을 사용하여, 필요없는 else를 지운다.

- 코드의 흐름을 더 단순화 하기 위해, Guard 절이 참이면 즉시 실행을 종료하는 코드로 리팩토링
- 이를 **Replace Nested Conditional with Guard Clauses** 라고 한다.

```
public class StringCalculator {  
  
    public static int splitAndSum(String text) {  
        int result = 0;  
  
        if (text == null || text.isEmpty()) {  
            result = 0;  
        }  
        else {  
            String[] values = text.split("-");  
            result += getSum(values);  
        }  
  
        return result;  
    }  
  
    private static int getSum(String[] values) {  
        int sum = 0;  
        for (String value : values) {  
            sum += Integer.parseInt(value);  
        }  
        return sum;  
    }  
}
```



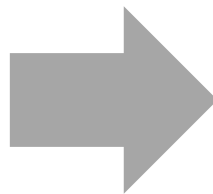
```
public class StringCalculator {  
  
    public static int splitAndSum(String text) {  
        int result = 0;  
  
        if (text == null || text.isEmpty()) {  
            return 0;  
        }  
  
        String[] values = text.split("-");  
        result += getSum(values);  
        return result;  
    }  
  
    private static int getSum(String[] values) {  
        int sum = 0;  
        for (String value : values) {  
            sum += Integer.parseInt(value);  
        }  
        return sum;  
    }  
}
```

하나의 역할을 하는 메소드로 만든다.

getSum 은 하나씩 수로 바꾸어 sum을 구하는 메서드

→ 수로 변경하는 메서드 / sum을 구하는 메서드로 분리

```
public class StringCalculator {  
  
    public static int splitAndSum(String text) {  
        int result = 0;  
  
        if (text == null || text.isEmpty()) {  
            return 0;  
        }  
  
        String[] values = text.split("-");  
        result += getSum(values);  
        return result;  
    }  
  
    private static int getSum(String[] values) {  
        int sum = 0;  
        for (String value : values) {  
            sum += Integer.parseInt(value);  
        }  
        return sum;  
    }  
}
```

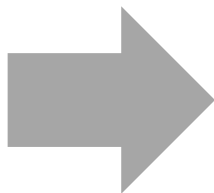


```
public class StringCalculator {  
  
    public static int splitAndSum(String text) {  
        int result = 0;  
  
        if (text == null || text.isEmpty()) {  
            return 0;  
        }  
  
        String[] values = text.split("-");  
        int[] numbers = toInts(values);  
        result = getSum(numbers);  
        return result;  
    }  
  
    private static int[] toInts(String[] values) {  
        int[] numbers = new int[values.length];  
  
        for (int i = 0; i < values.length; i++) {  
            numbers[i] = Integer.parseInt(values[i]);  
        }  
        return numbers;  
    }  
  
    private static int getSum(int[] numbers) {  
        int sum = 0;  
        for (int num : numbers) {  
            sum += num;  
        }  
        return sum;  
    }  
}
```

임시변수 제거하기

의미 파악에 도움이 되지 않는 변수 제거

```
public static int splitAndSum(String text) {  
    int result = 0;  
  
    if (text == null || text.isEmpty()) {  
        return 0;  
    }  
  
    String[] values = text.split("-");  
    int[] numbers = toInts(values);  
    result = getSum(numbers);  
    return result;  
}
```

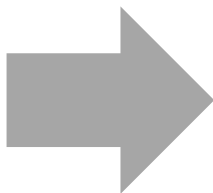


```
public static int splitAndSum(String text) {  
  
    if (text == null || text.isEmpty()) {  
        return 0;  
    }  
  
    return getSum(toInts(text.split("-")));  
}
```

추상화 Level을 맞추어준다.

더 이해하기 쉽게 변경

```
public static int splitAndSum(String text) {  
    if (text == null || text.isEmpty()) {  
        return 0;  
    }  
    return getSum(toInts(text.split("-")));  
}
```



```
public static int splitAndSum(String text) {  
    if (isEmpty(text)) return 0;  
    return getSum(toInts(split(text)));  
}
```

refactoring 완료

가장 중요한 메서드

public / private 구분
메서드 순서를 읽기 좋게 배치

```
public class StringCalculator {  
  
    public static int splitAndSum(String text) {  
        if (isEmpty(text)) return 0;  
        return getSum(toInts(split(text)));  
    }  
  
    private static String[] split(String text) {  
        return text.split("-");  
    }  
  
    private static boolean isEmpty(String text) {  
        if (text == null) return true;  
        return text.isEmpty();  
    }  
  
    private static int[] toInts(String[] values) {  
        int[] numbers = new int[values.length];  
  
        for (int i = 0; i < values.length; i++) {  
            numbers[i] = Integer.parseInt(values[i]);  
        }  
        return numbers;  
    }  
  
    private static int getSum(int[] numbers) {  
        int sum = 0;  
        for (int num : numbers) {  
            sum += num;  
        }  
        return sum;  
    }  
}
```

[도전] 직접 리팩토링 해보기

미션

1. else 없애기
2. indentation 레벨 줄이기
3. 메서드를 추가한다면,
하나의 역할만 하는 메서드로 추가



```
mincoding1 / splitAndSum.java Secret
Last active now

<> Code -> Revisions 2

리팩토링예제_Java버전

splitAndSum.java

1 public class Main {
2
3     public static int splitAndSum(String text) {
4         int result = 0;
5
6         if (text == null || text.isEmpty()) {
7             result = 0;
8         }
9         else {
10            String[] values = text.split("-");
11            for (String value : values) {
12                result += Integer.parseInt(value);
13            }
14        }
15    }
16 }
```

Chapter4

자주 사용되는 Refactoring 기법

자주 사용되는 리팩토링 행동들 살펴보기

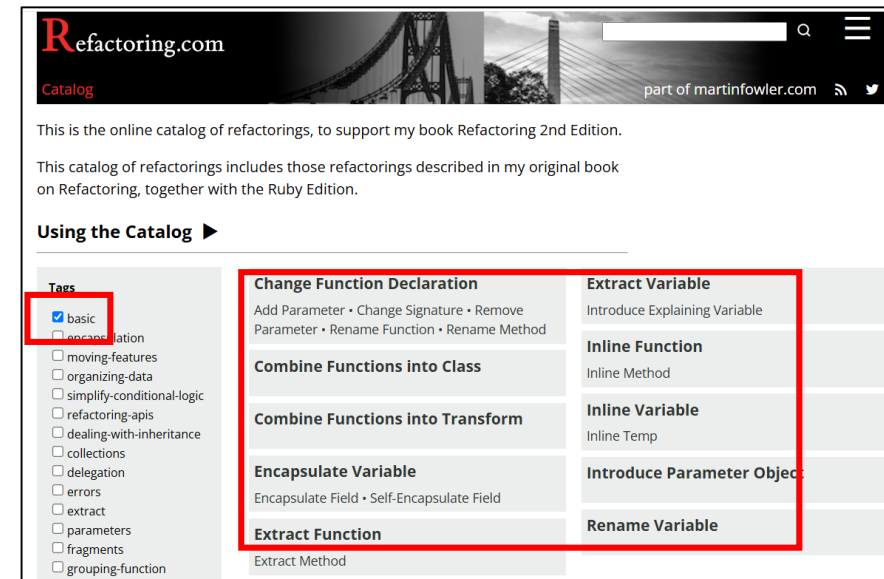
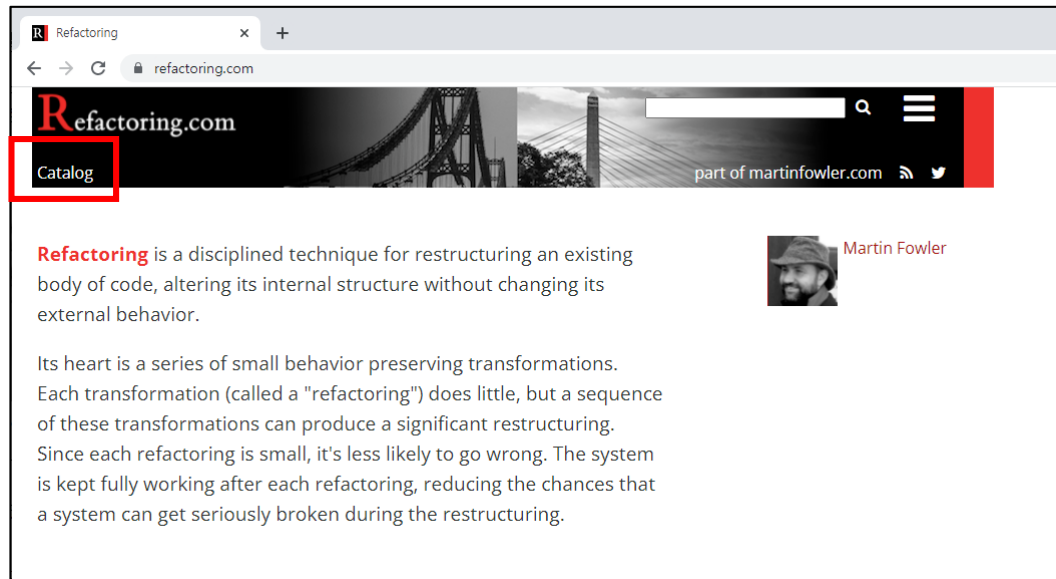
자주 사용되는 리팩토링 기법

가독성을 높이는데 다양한 리팩토링 행동들이 있다.

마틴 파울러는 코드를 수정할 때의 사소한 동작 하나하나에도 세심하게 이름을 붙였다.

이번 챕터에서는 가장 많이 사용되는 리팩토링 행동, 즉 "**리팩토링 기법**"에 대해 살펴본다.

(그 외 세부 기법들은 차후 수업에서 다룰 예정이다.)



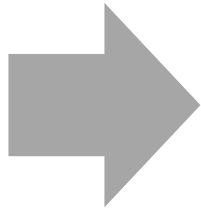
위 사이트에서 리팩토링 행동들의 이름을 확인할 수 있다.

<https://refactoring.com/catalog/>

Rename Variable

이름을 짓는데 공들여라.

```
int h = 10;  
int w = 20;  
int i = h * w;
```

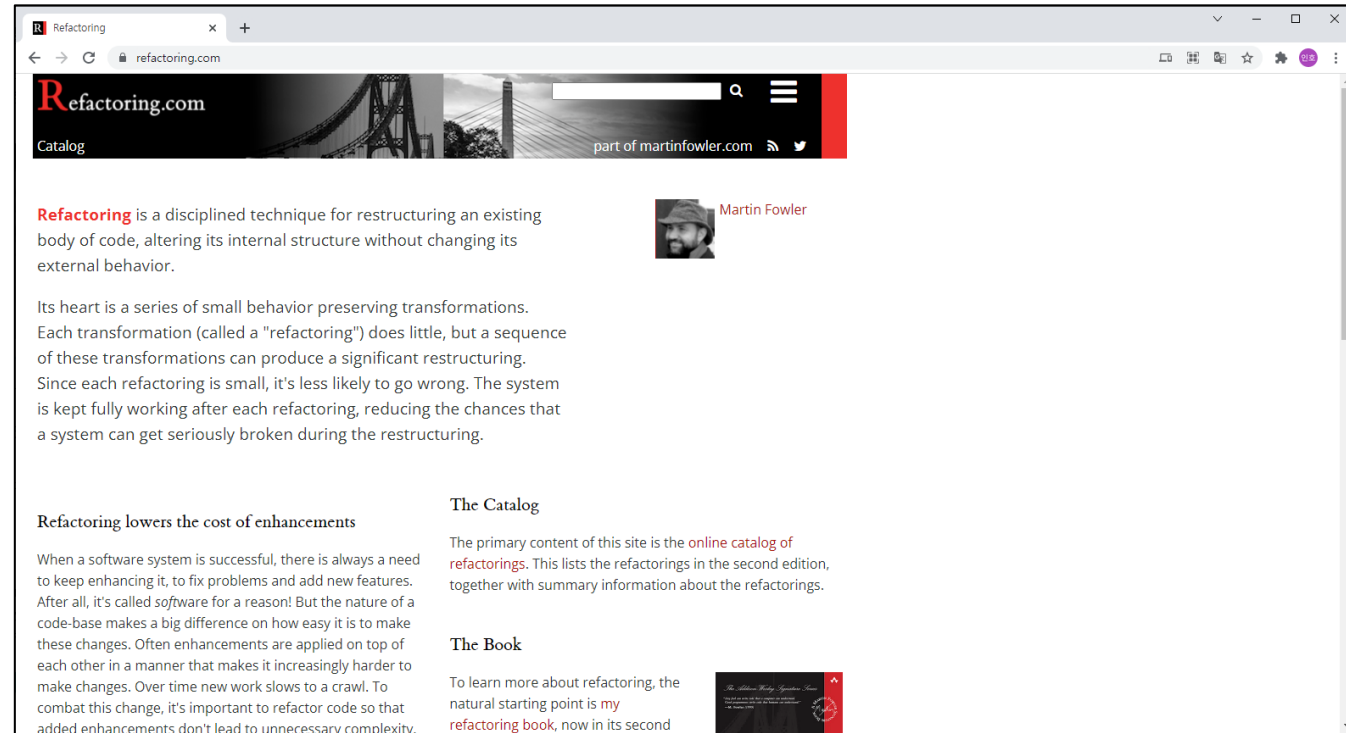


```
int height = 10;  
int width = 20;  
int area = height * width;
```

[참고] 마틴파울러 refactoring 사이트

소개되는 빨간색 이름들은, 마틴파울러가 정한 이름

- <https://refactoring.com/catalog/>



무슨 코드일까?

예측하기 힘든 코드

```
while(85 <= getTickCount()) {
```

Replace Magic Literal

빨간색 글씨
= 마틴파울러가 정한 이름

매직넘버 치환하기

→ 예측할 수 있는 코드

```
final int MAX_BOOTING_TICK_COUNT = 85;

while(MAX_BOOTING_TICK_COUNT <= getTickCount()) {

}
```

Refactoring.com
Catalog

Replace Magic Literal

$2 * 3.14 * \text{radius}$
 π

```
function potentialEnergy(mass, height) {  
  return mass * 9.81 * height;  
}
```

↓↓

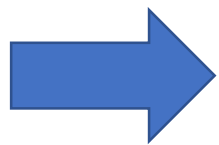
```
const STANDARD_GRAVITY = 9.81;  
function potentialEnergy(mass, height) {  
  return mass * STANDARD_GRAVITY * height;  
}
```

aliases Replace Magic Number with Symbolic Constant

상수 의존관계

상수끼리 관계가 있는 경우,
관계를 표시하도록 해주는 것이 가독성에 더 좋다.

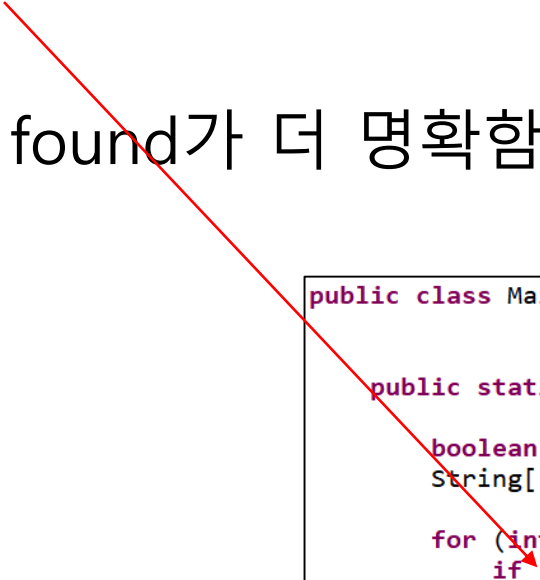
```
final int BOOT_AREA_KB = 200;  
final int HEAP_AREA_KB = 800;
```



```
final int BOOT_AREA_KB = 200;  
final int HEAP_AREA_KB = BOOT_AREA_KB * 4;
```

Rename Variable

flag 보다는 found가 더 명확함

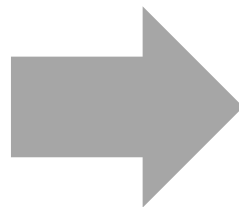


```
public class Main {  
  
    public static void main(String[] args) {  
  
        boolean flag = false;  
        String[] arr = {"ABC", "BBQ", "EER"};  
  
        for (int i = 0; i < arr.length; i++) {  
            if (arr[i] == "BBQ") {  
                flag = true;  
                break;  
            }  
        }  
  
        if (flag) System.out.println("있음");  
        else System.out.println("없음");  
    }  
}
```

Change Function Declaration

명확한 함수명 사용

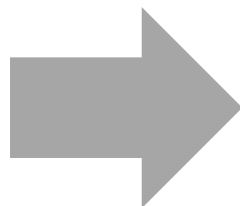
```
public class Cal {  
    int a;  
  
    int sum(int v) {  
        return a + v;  
    }  
}
```



```
public class Cal {  
    int totalCoin;  
  
    int accumulationCoin(int inputCoin) {  
        return totalCoin + inputCoin;  
    }  
}
```

Extract Function

```
void printAllData() {  
    //make data field  
    int[] vect = new int[10];  
  
    //cal acc value  
    int cnt = 0;  
    for (int i = 0; i<10; i++) {  
        vect[i] = cnt;  
        cnt += i;  
    }  
  
    //check exist data 10  
    int flag = 0;  
    for (int i = 0; i <10; i++) {  
        if (vect[i] == 10) {  
            flag = 1;  
            break;  
        }  
    }  
  
    //print result  
    if (flag == 1) {  
        System.out.println("발견");  
    }  
    else {  
        System.out.println("미발견");  
    }  
}
```

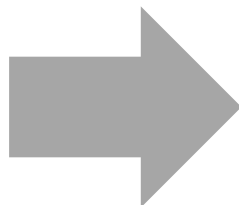


```
int[] makeDataField() {  
    return new int[10];  
}  
  
void calAccValue(int[] vect) {  
    int cnt = 0;  
    for (int i = 0; i<10; i++) {  
        vect[i] = cnt;  
        cnt += i;  
    }  
}  
  
boolean checkExistData(int[] vect, int tarNum) {  
    for (int i = 0; i <10; i++) {  
        if (vect[i] == 10) {  
            return true;  
        }  
    }  
    return false;  
}  
  
void printAllData() {  
    int[] vect = makeDataField();  
    calAccValue(vect);  
    if (checkExistData(vect, 10) == true) {  
        System.out.println("발견");  
    }  
    else {  
        System.out.println("미발견");  
    }  
}
```


Inline Function

필요하다면, 1회성 함수를 inline 시키기

```
int isCompare(int a, int b) {  
    if (a == b) return 1;  
    return 0;  
}  
  
boolean checkExistData(int[] vect, int tarNum) {  
    for (int i = 0; i < 10; i++) {  
        if (isCompare(vect[i], 10) == 1) {  
            return true;  
        }  
    }  
    return false;  
}
```



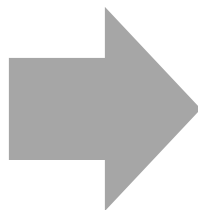
```
boolean checkExistData(int[] vect, int tarNum) {  
    for (int i = 0; i < 10; i++) {  
        if (vect[i] == 10) {  
            return true;  
        }  
    }  
    return false;  
}
```

Extract Variable 1

식을 이해하기 쉽게 하기 위해, 이름을 붙인다.

- **Introduce Explaining Variable** : 설명하기 위한 변수 도입하기

```
static double getCircleSize(Pack pack) {  
    return (((pack.DAend - pack.DAstart) / 2) * 0.5) + (((pack.DBend - pack.DBstart) / 2) * 0.5);  
}
```

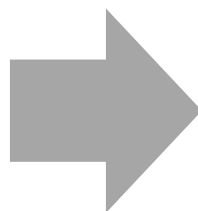


```
static double getCircleSize(Pack pack) {  
  
    double DAmid = (pack.DAend - pack.DAstart) / 2;  
    double DBmid = (pack.DBend - pack.DBstart) / 2;  
  
    double halfDAmid = DAmid * 0.5;  
    double halfDBmid = DBmid * 0.5;  
  
    return halfDAmid + halfDBmid;  
}
```

Extract Variable 2

의미를 이해하기 쉽다.

```
int getBestPrice(int[] dCode) {  
    if (dCode[0] + dCode[1] > dCode[2]) return dCode[2];  
    return dCode[0] + dCode[1];  
}
```

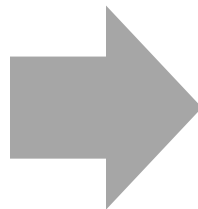


```
int getBestPrice(int[] dCode) {  
    int salePrice    = dCode[0] + dCode[1];  
    int specialPrice = dCode[2];  
  
    if (salePrice < specialPrice) return salePrice;  
    return specialPrice;  
}
```

Inline Variable

변수는 표현식에 이름을 붙여 의미를 이해하기 쉽게 만들지만, 이미, 의미가 충분하여 필요없는 경우도 있다.

```
boolean isBoxSizeMoreThan15(Pack box) {  
  
    int boxSize = box.size;  
    if (boxSize > 15) {  
        return true;  
    }  
    return false;  
}
```



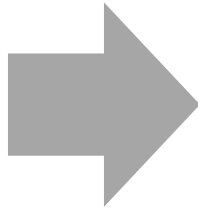
```
boolean isBoxSizeMoreThan15(Pack box) {  
    return box.size > 15;  
}
```

Encapsulate Variable

캡슐화를 할 것

- 최대한 객체의 필드는 private로 유지한다.
- 필드 값을 제어하는 통로를 줄인다.
- 유효값 검사 가능

```
Box box = new Box();  
box.size = 10;  
box.color = "BLUE";
```



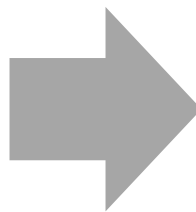
```
Box box = new Box();  
box.setSize(10);  
box.setColor("BLUE");
```

Introduce Parameter Object

파라미터 객체를 만들어 사용하기

- Introduce 라는 용어는 도입, 추출의 의미로 쓰인다. (inline의 반대 용어)

```
void drawPixel(int x, int y) {  
    //..  
}  
  
void drawCircle(int x, int y) {  
    //..  
}  
  
void drawTree(int x, int y) {  
    //..  
}
```



```
class Axis {  
    int x, y;  
}  
  
public class Main {  
    void drawPixel(Axis tar) {  
        //..  
    }  
  
    void drawCircle(Axis tar) {  
        //..  
    }  
  
    void drawTree(Axis tar) {  
        //..  
    }  
}
```

[도전] SumChecker KATA 리팩토링

본인의 스타일대로 리팩토링 시작!

가독성이 좋은 코드로 리팩토링

[리팩토링 훈련 규칙]

- else 최소화
- 아규먼트 최소화
- 들여쓰기 최소화
- 전역변수 사용 금지
- 작은 수정 마다 정상적으로 값 나오는지 테스트

[리팩토링] 연습 예제 2 Java

sumchecker.java

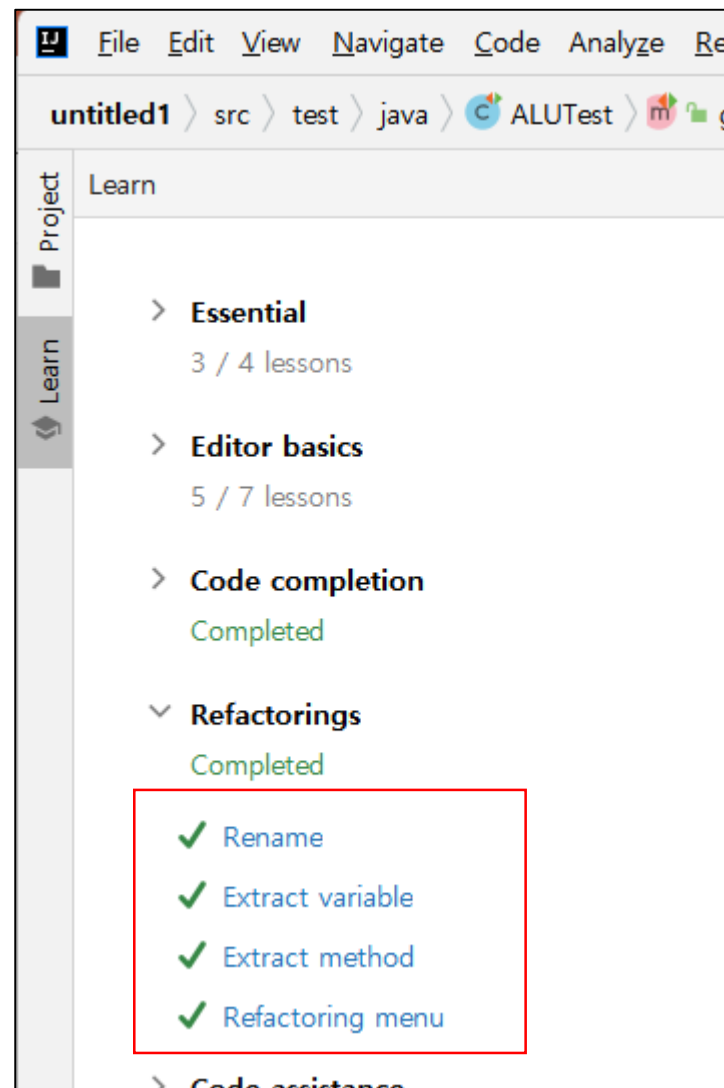
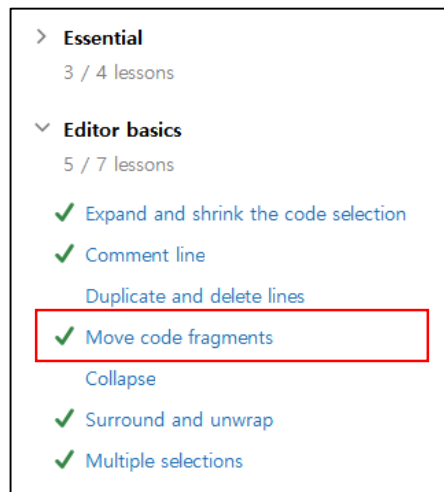
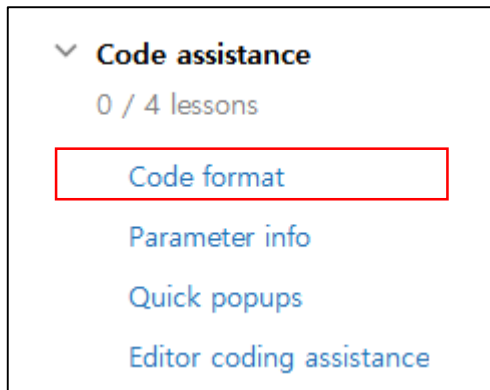
```
1 import java.util.ArrayList;
2 public class Main {
3     public static void main(String[] args) {
4         //25+61=100
5         //1 ~ 5자리수 덧셈 수식이 맞는지 확인하는 프로그램
6         //찍어쓰기 없음
7         String str = "25+61=86"; //PASS
8         //String str = "12345+12345=24690"; //PASS
9         //String str = "5++5=10"; //ERROR
10        //String str = "10000+1=10002"; //FAIL
11
12        System.out.println(getResult(str));
13    }
14    private static String getResult(String str) {
15        int flag1 = 0;
16        int cnt1 = 0;
17        int cnt2 = 0;
18        int p1 = 0, p2 = 0;
19        //+=와 = 개수 확인
20        for (int i = 0; i < str.length(); i++) {
21            if (str.charAt(i) == '+') {
22                cnt1++;
23                p1 = i;
24            }
25            else if (str.charAt(i) == '=') {
26                cnt2++;
27                p2 = i;
28            }
29            else if (!(str.charAt(i) >= '0' && str.charAt(i) <= '9')) {
30                flag1 = 1;
31                break;
32            }
33        }
34        int flag2 = 0;
35        if (cnt1 == 1 && cnt2 == 1 && p1 < p2 - 1 && flag1 == 0) {
```

<https://gist.github.com/mincoding1/733a88524da62cea06063d80fa1313ea>

[참고] 단축키 가이드

Refactoring 을 위한 단축키 자습서

- Rename : 단축키 암기
- Refactoring Menu : 단축키 암기
 - Extract variable / method 는 단축키 암기 안해도 됨



Chapter5

리팩토링시 고려할 점

팀원들을 고려한 리팩토링

리팩토링을 할 때 고려할 점

리팩토링을 적절히 수행하면 코드의 품질이 올라간다.

가독성이 증가

→ 유지보수가 편리

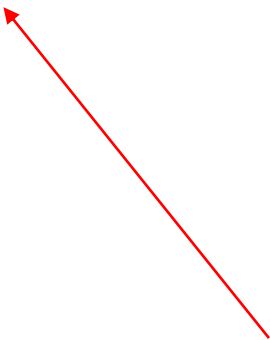
→ 개발속도 증가

무조건적인 리팩토링을 하기 전,
고려해야 할 점들이 존재한다.

자신이 리팩토링을 하는 이유를 정확히 구분하자. (메타인지)

오버엔지니어링 (본인의 만족)
정말로 팀에 필요한 리팩토링인지

확장성 / 생산성을 과하게 고려하여
코드 복잡성 / 개발시간을 높인다.



팀원들의 감정

누군가 허락없이 내가 짠 코드를 수정하면 감정 문제로 퍼진다.
트러블 / 작은 감정문제는 생산성 하락을 초래한다.

레거시 코드에는 의도가 있다.

- 코드만 보고, 소스코드의 모든 의도를 파악하는 것은 어렵다.
- 의도가 불분명한 코드는 코드 개발자와 대화 필요

버전관리도구, Test 환경이 충분히 갖춰져야 한다.

리팩토링 수행 전 후 결과에 대한 Unit Test / Black Box Test 등 환경이 갖춰져야 한다.

개발중이 아닌, 개발이 완료되어 동작되는 모듈에 대해 리팩토링을 한다.

버그가 거의 없어야 한다.

→ 버그가 너무 많은 프로그램은 리팩토링 작업이 힘들다.